

La memoria y la forma de trabajar

Cómo está organizada la memoria del repositorio, cómo se reparte el trabajo para que dos personas y sus agentes no se encimen, y por qué trabajamos en sesiones cortas con el estado en el repo y no en el chat.

PARA	COMPLEMENTA	FECHA
el segundo dev y su agente	Manual · La rampa TRX	2026-09-07 · v1

Parte I · La memoria del repositorio

Un grafo de documentos que se recorre a demanda, no un manual que se lee entero.

01 Por qué existe

En julio, un documento interno ya decía que una racha de resultados del bot era un espejismo estadístico. El repositorio **lo sabía**. El código siguió publicando el número bonito durante un mes, hasta que se descubrió que casi dos tercios de esos cierres se habían escrito en 763 milisegundos por un proceso mal configurado.

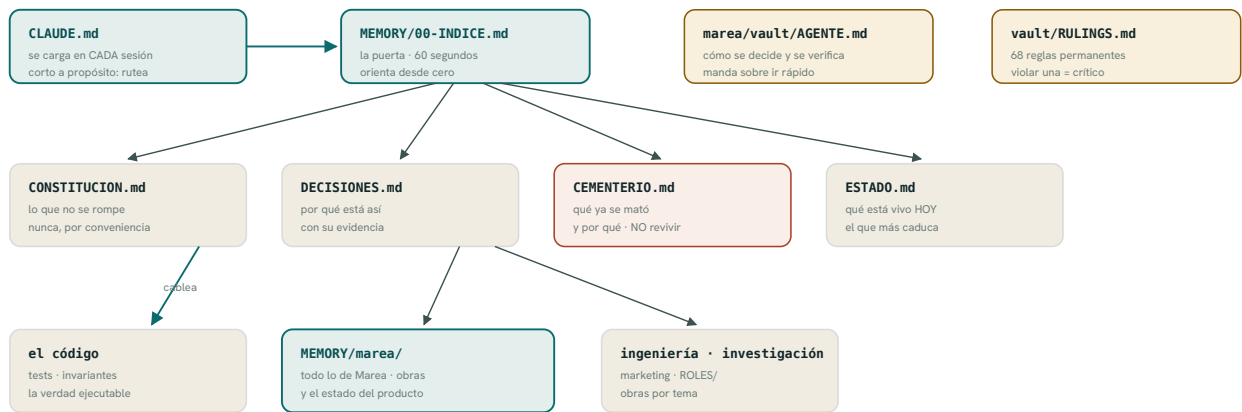
LA LECCIÓN, Y LA REGLA QUE SALIÓ DE ELLA

El fallo no fue de conocimiento sino de cableado. El saber estaba escrito y no cambió nada, porque nada lo hacía cumplir. De ahí la regla que gobierna todo lo demás:

Un hallazgo sin un test que lo haga cumplir es una nota, no un arreglo. Cuando cierres algo, pregúntate qué prueba impide que vuelva. Si la respuesta es «acordarse», no está cerrado.

La memoria existe por el otro lado del mismo problema: **los agentes no se descarrilan por falta de inteligencia, sino por falta de contexto.** Olvidan por qué se decidió algo, qué está prohibido, qué ya se probó y murió. `MEMORY/` es la bitácora durable que cierra ese hueco.

02 El mapa



Regla del grafo: se rutea, no se repite.

Si dos nodos dicen lo mismo, uno de los dos va a mentir eventualmente — y no vas a saber cuál.

03 Cómo se lee: rutas de entrada

No se lee entero. Se entra por la ruta de tu tarea. Todo nodo se alcanza en dos saltos desde el índice.

SI VAS A...	LEES
Empezar cualquier cosa	CLAUDE.md → MEMORY/00-INDICE.md
Tocar el motor, una config o una invariante	MEMORY/CONSTITUCION.md
Entender por qué algo está así	MEMORY/DECISIONES.md
Proponer una idea	MEMORY/CEMENTERIO.md — primero, no después
Saber qué está vivo o pendiente hoy	MEMORY/ESTADO.md
Trabajar en Marea	marea/vault/AGENTE.md + RULINGS.md + MEMORY/marea/README.md
Tu encargo	marea/vault/ENCARGO_RAMPA.md

04 Cómo se escribe

- 01 Un nodo, una responsabilidad.** Una decisión va a DECISIONES. Una idea muerta va al CEMENTERIO. El estado de hoy va a ESTADO. No se mezclan.
- 02 Nada se duplica.** Si necesitas repetir algo, enlázalo por nombre de archivo para que un agente pueda abrirlo.
- 03 Todo nodo lleva fecha.** ESTADO es el que más caduca: si la fecha es vieja, confírmalo contra `git log` antes de crearle.
- 04 El cementerio es de primera clase.** Saber qué NO hacer vale tanto como saber qué hacer, y se escribe con la razón, no sólo el veredicto.
- 05 Se escribe al cerrar, no al empezar.** Cuando termines algo que costó descubrir, déjalo en `MEMORY/marea/` con fecha. Ése es el momento en que sabes lo que aprendiste.
- 06 Documentar no es entregar.** Un documento sólo cuenta si cambia una decisión de alguien. Si no, es ruido con formato.

Parte II • Cómo no encimarnos

Dos personas y dos agentes sobre el mismo repositorio. El reparto es por archivo, no por buena voluntad.

05 Dueños por zona

Durante tu encargo, cada archivo tiene un dueño. El dueño edita; el otro pregunta.

TUYA • <code>adapters/puente/*</code>	La rampa entera. Nadie más la toca.
TUYA • <code>domain/solicitudes.ts</code>	Se extiende , no se reescribe: la máquina de estados ya está probada.
TUYA • <code>screens de depósito</code>	Pantallas y hojas del flujo de entrada de dinero.
NUESTRA • <code>domain/pozo.ts</code>	El compensador. Invariantes L1, L2 y L5.
NUESTRA • <code>domain/parimutuel.ts</code>	El reparto y el multiplicador. Cambio en curso: la semilla pasa a subsidio.
NUESTRA • <code>contratos/</code> • <code>merkle</code> • <code>época</code>	La parte en cadena y el anclaje.
COMPARTIDA • <code>adapters/index.ts</code>	Donde se cablean todos los adapters. Aquí chocamos.
COMPARTIDA • <code>lib/config.ts</code>	Variables de entorno de todas las integraciones.
COMPARTIDA • <code>lib/strings.ts</code>	Todo el texto de la app.
COMPARTIDA • <code>state/store.tsx</code>	El store único.

PROTOCOLO DE LA ZONA COMPARTIDA

Son cuatro archivos y son el 90% de los conflictos. Ahí: **commits pequeños** (una razón por commit) • **traer main a tu rama a diario**, no cada dos semanas • **añadir al final**, no reordenar bloques existentes • y **avisar antes** si vas a mover algo estructural. Un conflicto en ``strings.ts`` es trivial; uno en ``store.tsx`` cuesta media tarde.

06 Ramas y fusiones

- 01 Una rama por encargo, con nombre descriptivo. La tuya para esto: `claude/marea-rampa-tron`.
- 02 Fusiona tus propias ramas cuando quieras. Entre ramas tuyas haz lo que te sirva — merge, rebase, squash. Es tu espacio.
- 03 Nunca reescribas historia de una rama compartida. Si alguien ya la sacó, un rebase le rompe el checkout. En rama compartida, sólo merge.
- 04 A ``main`` sólo por PR. ``main`` redespiega producción del bot. No es una convención de estilo: es un despliegue.

05 Trae ``main`` a tu rama, no al revés. A diario. La divergencia se paga con interés compuesto.

06 Antes de empezar el día: `git fetch --all` y una mirada a las ramas activas. Treinta segundos que evitan reescribir lo que alguien ya hizo.

Parte III • Ingeniería agéntica

Cómo se trabaja con un agente sin quemar presupuesto ni perder el hilo.

07 Lo que de verdad cuesta un turno

Conviene entender el mecanismo antes que la receta, porque la receta ingenua está mal.

La API no tiene estado. Cada turno se reenvía la conversación entera. Así que el coste de un turno no es lo que escribes: es **todo lo que arrastras**.

Contra eso existe el **caché de prefijo**, y sus reglas mandan sobre todo lo demás:

Leer del caché, contra pagar entrada completa	≈ 10%
Escribir al caché, la primera vez	≈ 1.25×
Vida del caché por defecto	5 minutos
Vida ampliada, si se pide	1 hora
Cualquier byte que cambie en el prefijo	invalida todo lo que sigue

De ahí sale la conclusión que contradice la intuición: una sesión larga **no** es cara por ser larga. Si el prefijo se mantiene estable, continuar cuesta una décima parte. Lo caro son otras dos cosas — **invalidar el caché** (tocar algo del principio y hacer que todo se vuelva a pagar completo) y **arrastrar contexto irrelevante**, que se paga dos veces: en tokens y en que el modelo trabaja peor rodeado de ruido.

08 Cuándo reiniciar en frío, y cuándo no

LA REGLA

Reinicia cuando el contexto acumulado dejó de ser relevante — no cuando es largo.

Si llevas dos horas en el mismo archivo y el hilo entero es útil, seguir es barato. Si cambiaste de tarea y arrastras cuarenta turnos de otra cosa, reinicia: estás pagando por transportar basura y además el modelo la está leyendo.

Y lo que hace barato reiniciar es la memoria. Un arranque en frío paga precio completo por lo que cargues — así que la pregunta es cuánto necesitas cargar. Con el estado en el repositorio, arrancar cuesta **tres archivos**: `CLAUDE.md`, el documento de tu encargo, y `ESTADO.md`. Sin eso, cuesta cuarenta turnos de historia que alguien tiene que reconstruir de memoria.

POR ESO LA PARTE I Y LA PARTE III SON EL MISMO TEMA

La memoria del repositorio no es orden por gusto: **es lo que hace que una sesión desechable no pierda nada**. El repo es la memoria; la conversación es material de trabajo. Si una decisión sólo vive en el chat, se perdió — aunque el chat siga abierto.

Señales de que hay que reiniciar

- ▶ El agente vuelve a leer archivos que ya leyó en esa misma sesión.
- ▶ Repite conclusiones a las que ya había llegado, o se contradice con algo de hace veinte turnos.
- ▶ Cambiaste de tarea. Ésta es la más común y la más fácil de ignorar.
- ▶ Vas a hacer algo largo y el hilo previo no aporta nada a ello.
- ▶ Volviste después de un rato largo: con cinco minutos de vida por defecto, el caché ya está frío de todos modos.

Anti-patrones

- ✗ **Usar el chat como memoria.** Es el error caro. Lo que importe va al repo.
- ✗ **Reiniciar cada quince minutos «por ahorrar».** Cada arranque paga precio completo por lo que cargues; si el contexto sigue siendo útil, reiniciar es más caro, no menos.
- ✗ **Meter la hora, un id aleatorio o un JSON desordenado al principio del contexto.** Cambia el prefijo en cada turno y mata el caché sin que se note.

09 El ciclo de una sesión

ARRANQUE (en frío, tres lecturas)

- 1 CLAUDE.md ruta · 60 s
- 2 el doc de tu encargo qué hay que hacer y su puerta de cierre
- 3 ESTADO.md qué está vivo hoy · git fetch

TRABAJO (una unidad, con puerta de cierre)

- 4 una sola cosa por sesión
- 5 el test se escribe en el mismo commit que el código
- 6 rómpelo a propósito y confirma que se pone rojo
- 7 npm run ci antes de commitear

CIERRE (lo que hace desechable la conversación)

- 8 commit con el PORQUÉ en el mensaje
- 9 lo que costó descubrir → MEMORY/marea/ con fecha
- 10 cierra la sesión. La siguiente arranca en frío y no pierde nada.

10 Las siete prácticas de la casa

- 01 **Cierra el lazo:** hallazgo → test → invariante. Es la regla de la que salió todo el repositorio.
- 02 **Encargos con puerta, no tareas abiertas.** Cada unidad de trabajo define cómo se sabe que terminó, *antes* de empezarla.
- 03 **No le pidas al agente lo que un test puede verificar.** «Revisa que no rompí nada» es una prueba, no una petición.
- 04 **Rompe tu propia prueba a propósito.** Una prueba que nunca viste en rojo no sabes si prueba algo.
- 05 **Una métrica demasiado limpia es un bug.** Si todo pasa a la primera y cubre mucho, sospecha de la medición antes que celebrar el código.
- 06 **El porqué va en el commit.** El diff ya dice qué cambió; nadie puede reconstruir por qué.
- 07 **Prefiere editar a crear.** Duplicar es deuda, y en un grafo de documentos es peor: dos nodos que dicen lo mismo terminan contradiciéndose.

11 Caso de estudio: cómo se hizo **pozo.ts**

Un ejemplo real de esta semana, porque enseña más que las siete reglas juntas.

- 1 Se escribió el módulo puro y su prueba de propiedad: mil secuencias aleatorias de operaciones, liquidando cada pozo con cada ganador posible. **Verde a la primera.**
- 2 Verde a la primera es sospechoso (práctica 05). Se rompió el código **a propósito tres veces** — quitar una validación, pagar 1% de más, no quemar contratos al fusionar. Las tres se pusieron rojas. La prueba tenía dientes.
- 3 Se midió el perfil del generador para ver qué cubría de verdad. **Y ahí saltó:** cero mercados de tres resultados. El generador de números aleatorios, sembrado con valores consecutivos, daba primeras

extracciones correlacionadas — y la primera extracción era justo la que decidía cuántos resultados tenía el mercado.

- 4 **Mil secuencias en verde que nunca probaron un caso entero.** Se corrigió, se volvió a medir (331 / 333 / 336), y el porqué quedó escrito en el código para que nadie lo «simplifique» de vuelta.

LO QUE HAY QUE LLEVARSE

El bug no estaba en el código que se probaba: **estaba en la prueba**. Y no lo encontró el agente escribiendo más código — lo encontró **medir si la prueba cubría lo que decía cubrir**. Ésa es la práctica que más rinde de todas.

A Anexo • Las dos listas

Al abrir sesión

- ☐ `git fetch --all` y mirar ramas activas
- ☐ Leer `CLAUDE.md`
- ☐ Leer el doc del encargo
- ☐ Leer `ESTADO.md` y comprobar su fecha
- ☐ Si vas a proponer algo: `CEMENTERIO.md`
- ☐ Elegir **una** unidad de trabajo y su puerta

Al cerrar sesión

- ☐ La prueba existe y la viste en rojo una vez
- ☐ `npm run ci` verde
- ☐ Commit con el porqué
- ☐ Lo aprendido → `MEMORY/marea/` con fecha
- ☐ Rama empujada
- ☐ Cerrar la conversación sin miedo